

A PROJECT REPORT ON

**Improving the efficiency of Deep Learning Based
Long Document Classification**

**SUBMITTED TO THE SAVITRIBAI PHULE UNIVERSITY, PUNE
IN THE PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE AWARD OF THE DEGREE**

OF

BACHELOR OF ENGINEERING (Computer Engineering)

SUBMITTED BY

Prathamesh Kokate
Manavi Khopade
Mitali Sarnaik

Exam No: B400050452
Exam No: B400050497
Exam No: B400050580



DEPARTMENT OF COMPUTER ENGINEERING
Pune Institute of Computer Technology
Dhankawadi, Pune - 411043

SAVITRIBAI PHULE PUNE UNIVERSITY
2024-2025



CERTIFICATE

This is to certify that the project report entitled
Improving the efficiency of Deep Learning Based Long Document Classification

Submitted by

Prathamesh Kokate
Manavi Khopade
Mitali Sarnaik

Exam No: B400050452
Exam No: B400050497
Exam No: B400050580

is a bonafide student of this institute and the work has been carried out by him/her under the supervision of **Dr. M.S. Takalikar** and it is approved for the partial fulfillment of the requirement of Savitribai Phule Pune University, for the award of the degree of **Bachelor of Engineering** (Computer Engineering).

Dr. M. S. Takalikar
Internal Guide
Dept. of Computer Engg.

Dr. G. V. Kale
Head
Dept. of Computer Engg.

Dr. S. T. Gandhe
Principal
Pune Institute of Computer Technology

Place:Pune
Date:

ACKNOWLEDGEMENT

*It gives us great pleasure in presenting the project report on ‘**Improving the Efficiency of Deep Learning Based Long Document Classification**’.*

*We would like to take this opportunity to thank **Dr. M.S.Takalikar** giving us all the help and guidance we needed. We are really grateful to them for their kind support. Their valuable suggestions were very helpful.*

*We are also grateful to **Dr. G.V. Kale**, Head of Computer Engineering Department, PICT for her indispensable support, suggestions.*

*In the end our special thanks to **Mr. Raviraj Joshi**, L3Cube Labs, and all faculty members for their whole hearted cooperation for completion of this report. We also thank our laboratory assistants for their valuable help in laboratory.*

Prathamesh Kokate
Manavi Khopade
Mitali Sarnaik
(B.E. Computer Engg.)

ABSTRACT

Classifying long documents efficiently remains challenging due to fixed input limitations and high computational costs of transformer models, especially in cross-domain adaptation from Short Headline Classification (SHC) to Long Document Classification (LDC) tasks. We propose a sentence ranking approach using Term Frequency–Inverse Document Frequency (TF-IDF) to extract the most informative sentences, reducing input size while preserving classification accuracy.

We explore multiple sentence selection strategies, including truncation, random selection, and TF-IDF-based ranking. Experiments show that sentences at the beginning of documents are generally more informative, while random selection yields inconsistent results. Ranking sentences achieves a strong balance between context size and accuracy, with optimal key sentence selection enhancing document representation. Removing less informative sentences further boosts performance.

Using the MahaNews dataset and the marathi-bert-v2 model, we demonstrate that SHC-to-LDC adaptation through ranked sentence selection effectively reduces computational overhead while maintaining robust classification accuracy.

Contents

1	Introduction	1
1.1	Overview	2
1.2	Motivation	2
1.3	Problem Definition and Objectives	2
1.4	Project Scope & Limitations	3
1.5	Methodologies of Problem solving	3
2	Literature Survey	5
2.1	Datasets, Models, and Techniques for Long Document Classification	6
2.1.1	Datasets for Long Document Classification	6
2.1.2	Models for Long Document Classification	7
2.1.3	Techniques for Efficiency Improvement	7
3	Software Requirements Specification	10
3.1	Assumptions and Dependencies	11
3.2	Functional Requirements	12
3.2.1	System Feature 1 (Long Document Classification)	12
3.2.2	System Feature 2 (Pre-Trained Model Utilization)	12
3.3	External Interface Requirements	13
3.3.1	User Interfaces	13
3.3.2	Hardware Interfaces	13
3.3.3	Software Interfaces	13
3.3.4	Communication Interfaces	13
3.4	Nonfunctional Requirements	14
3.4.1	Performance Requirements	14
3.4.2	Safety Requirements	14
3.4.3	Security Requirements	14
3.4.4	Software Quality Attributes	14
3.5	System Requirements	15
3.5.1	Database Requirements	15

3.5.2	Software Requirements (Platform Choice)	15
3.5.3	Hardware Requirements	15
3.6	Analysis Models: SDLC Model to be applied	15
4	System Design	16
4.1	System Architecture	17
4.2	Mathematical Model	17
4.2.1	Sentence Scoring Mechanism	17
4.2.2	Sentence Ranking and Selection	18
4.2.3	Advantages Over Baseline Methods	18
4.3	Data Flow Diagrams	19
4.3.1	Level 0 DFD	19
4.3.2	Level 1 DFD	19
4.4	Entity Relationship Diagram	20
4.5	UML Diagrams	21
4.6	Sequence Diagram	23
5	Project Plan	24
5.1	Project Estimate	25
5.1.1	Reconciled Estimates	25
5.1.2	Project Resources	25
5.2	Risk Management	26
5.2.1	Risk Identification	26
5.2.2	Risk Analysis	26
5.2.3	Overview of Risk Mitigation, Monitoring, Management	27
5.3	Project Schedule	28
5.3.1	Project Task Set	28
5.3.2	Task Network	28
5.3.3	Timeline Chart	29
5.4	Team Organization	29
5.4.1	Team structure	29
5.4.2	Management reporting and communication	29
6	Project Implementation	31
6.1	Overview of Project Modules	32
6.2	Tools and Technologies Used	36
6.3	Algorithm Details	37
6.3.1	Document Classification	37
6.3.2	Sentence Selection	38

7	Software Testing	39
7.1	Type of Testing	40
7.2	Test cases & Test Results	41
8	Results	42
8.1	Outcomes	43
8.1.1	Model Accuracy	43
8.1.2	Inference Time	45
8.2	System Screen Shots	46
9	Conclusions	47
9.1	Technical Contributions	48
9.2	Limitations and Challenges	49
9.3	Future Work	50
9.4	Applications	51
10	Appendix	53
10.1	Appendix A	54
10.2	Appendix B	56
10.3	Appendix C	57
11	References	58

List of Figures

4.1	System Architecture	17
4.2	Level 0 Data Flow Diagram	19
4.3	Level 1 Data Flow Diagram	19
4.4	Entity Relationship diagram	20
4.5	Use case diagram	21
4.6	Activity Diagram	22
4.7	Sequence Diagram	23
5.1	Task Network	28
5.2	Timeline Chart	29
6.1	Sentence Selection Diagram	33
6.2	Process Diagram	34
8.1	User Interface	46
10.1	Plagiarism Report	57

CHAPTER 1

INTRODUCTION

1.1 Overview

Long document classification is a critical task in natural language processing (NLP), with applications in diverse domains such as categorizing research papers, legal documents, news articles, and customer reviews. Recent advancements in transformer-based models like BERT and its variants have set new benchmarks for classification accuracy. However, these models are inherently limited by their fixed input size and the quadratic complexity of their attention mechanism, making them inefficient when handling lengthy documents.

1.2 Motivation

Long document classification is a critical task in natural language processing (NLP), with applications in diverse domains such as categorizing research papers, legal documents, news articles, and customer reviews. Recent advancements in transformer-based models like BERT and its variants have set new benchmarks for classification accuracy. However, these models are inherently limited by their fixed input size and the quadratic complexity of their attention mechanism, making them inefficient when handling lengthy documents.

1.3 Problem Definition and Objectives

The primary objective of this project is to improve the efficiency of deep learning-based long document classification models without modifying the underlying transformer architecture. We aim to achieve this by optimizing the input representation using a TF-IDF-based sentence ranking approach. Specifically, the project seeks to:

- Reduce input length by selecting the most relevant sentences.
- Preserve contextual richness required for accurate classification.
- Minimize computational overhead during inference.
- Evaluate different sentence selection strategies and their impact on classification accuracy.

1.4 Project Scope & Limitations

Project Scope:

This project focuses on enhancing the efficiency of long document classification by optimizing the input representation for transformer-based models, specifically **marathi-bert-v2**. Rather than modifying the underlying architecture or training large-scale custom models, we propose a lightweight, data-driven technique using **TF-IDF-based sentence ranking**. The approach involves selecting the most informative sentences from a document to reduce input length while retaining essential contextual information. All experiments are performed on the **MahaNews dataset**, which comprises long Marathi news articles labeled by topic.

Limitations:

Although the proposed method effectively reduces inference time and maintains classification accuracy, it comes with certain limitations:

- The TF-IDF-based ranking assumes that important content correlates with word frequency and uniqueness, which may not always align with semantic importance.
- The strategy is currently evaluated only on the **MahaNews** dataset and may require adaptation for other datasets, domains, or languages.
- Performance may degrade for documents where context is distributed evenly across the text or heavily relies on discourse structure.
- The method does not handle multi-modal content (e.g., tables, figures) and assumes clean, sentence-level segmentation.

1.5 Methodologies of Problem solving

TF-IDF-Based Sentence Ranking:

To efficiently classify long documents using transformer-based models with limited input length, we propose a TF-IDF-based sentence ranking mechanism for input optimization. The primary goal of this method is to retain the most relevant contextual information while reducing redundant or less important content from the document. This enables the use of standard models like **marathi-bert-v2** without architectural modifications.

Each document is segmented into individual sentences, and each sentence is treated as a separate unit. We compute the TF-IDF (Term Frequency–Inverse Document Frequency) score of each word within a sentence,

treating each sentence in a document as a separate "document" for the purpose of calculation. The sentence-level score is then obtained by summing the TF-IDF scores of all words in that sentence. Higher cumulative scores indicate greater relevance, allowing for effective ranking of sentences in descending order of informativeness.

Sentence Selection Strategies:

To assess the effectiveness of our ranking mechanism, we explore and evaluate multiple strategies for selecting sentences based on their TF-IDF ranks:

- **Fixed-length Selection:** This method involves selecting a predefined number of top-ranked sentences from each document. Experiments are conducted with selections of the top 1, 2, 3, 4, and 5 sentences. This strategy helps determine the minimum number of sentences required to retain sufficient context for accurate classification.
- **Percentage-based Selection:** Instead of fixing a count, this strategy selects a specific percentage of top-ranked sentences from each document. The percentages evaluated include 25%, 50%, 75%, and 100%. This provides a flexible trade-off between compression and context preservation.
- **Weighted Ranking:** In this approach, we enhance the basic TF-IDF score by incorporating sentence length as an additional factor. This helps balance between high-scoring but short sentences and moderately-scoring longer sentences that may contain more detailed information. Different weighting formulas are explored to identify the optimal combination of TF-IDF relevance and content richness.

Implementation and Evaluation:

We apply the above strategies to preprocess the **MahaNews** dataset, generating reduced-context versions of each article. These modified inputs are then used to train and evaluate a **marathi-bert-v2** classification model. The performance of each selection strategy is compared in terms of classification accuracy and inference time, against baseline approaches such as selecting the first k sentences, last k sentences, or random sentence selection.

Our methodology demonstrates that TF-IDF-based sentence ranking, particularly when combined with length-aware weighting, results in significant improvements in both efficiency and accuracy over naive truncation or random sampling methods.

CHAPTER 2

LITERATURE SURVEY

2.1 Datasets, Models, and Techniques for Long Document Classification

2.1.1 Datasets for Long Document Classification

Datasets play a crucial role in the development and evaluation of long document classification models, providing the necessary data to benchmark model performance and scalability [13]. The complexity and diversity of datasets are key to ensuring that models are robust and capable of handling real-world challenges. Large-scale datasets, such as those covering legal texts, scientific articles, and multilingual news, are particularly important for testing the efficiency of models in different domains. These datasets not only present challenges due to their length and variability but also help identify how well models can manage intricate document structures [14], making them essential for advancing long document classification techniques.

The EURLEX-57K dataset, a European Union legal text collection, is a comprehensive resource containing long documents related to legislation and requiring multi-label classification [1,4]. Similarly, PubMed Abstracts, a biomedical dataset, offers long abstracts of scientific articles, often used for multiclass or multi-label classification [2]. The arXiv Papers dataset, consisting of full-length research papers, presents a challenging task for document classification due to their length and complexity [2]. The GovReport dataset, with government reports often exceeding 10,000 tokens per document, is large-scale and used for abstractive summarization and classification [5]. L3Cube-MahaNews, a Marathi news classification dataset, is available in three formats—short, medium, and long text classifications—and contains over 100,000 records sourced from a well-known Marathi news website [15]. This dataset is invaluable for comparing machine-learning algorithms in low-resource environments, covering 12 diverse categories. The L3Cube-IndicNews dataset is a versatile resource for classifying short headlines, long documents, and paragraphs in 10 major Indian languages, providing high-quality news articles from reliable sources, with no length restrictions [16]. These datasets, with their inherent complexity, present significant challenges that make them ideal for testing long document classification models.

2.1.2 Models for Long Document Classification

In terms of models, transformers and specialized techniques are designed to improve processing efficiency for long-form text. BERT, or Bidirectional Encoder Representations from Transformers, is foundational for many natural language processing tasks, capturing context effectively through its transformer architecture. Its bidirectional training allows a deeper understanding of sentence structures and semantic relationships, making it a popular choice for various classification challenges. A hybrid model that combines BERT with TextRank enhances classification efficiency by summarizing content before classification. This reduces input size while retaining key information, making it particularly effective for lengthy texts [12]. The BERT + Random model takes a simpler approach by sampling random sections from a document for classification, aiming to mitigate the computational load associated with long documents, though it may overlook some contextual cues [12]. Longformer, designed specifically for long documents, enhances the transformer framework by combining local and global attention mechanisms. This model efficiently processes longer input sequences while maintaining contextual relationships across the document [12]. CogLTX, another model tailored for long document classification, uses selective attention to focus on key sections and reduce computational complexity. While it performs well, it demands high GPU memory and training time, making it less efficient than models like Mamba or BERT variants [12]. Mamba, which leverages hierarchical attention mechanisms, is designed to enhance both performance and efficiency by targeting critical sections of the document during classification [4]. Other models include multilingual BERT models like MahaBERT, trained on 752 million tokens, and IndicBERT, which is pre-trained on 12 Indian languages with the AI4Bharat IndicNLP Corpora, comprising approximately 9 billion tokens [15,16,17]. LNLf-BERT is another advanced bidirectional transformer, applying attention at both the sentence and document levels using hierarchical attention with special [CLS] tokens [18].

2.1.3 Techniques for Efficiency Improvement

Techniques aimed at improving the efficiency of long document classification models primarily focus on reducing computational complexity and memory requirements. Sparse attention mechanisms are one such technique, with BIGBIRD being a notable example. BIGBIRD addresses inefficiencies in traditional transformer architectures by employing a sparse attention mechanism that reduces memory and computation costs from quadratic to linear. This allows the model to handle significantly longer inputs while retaining the

benefits of full-attention models. BIGBIRD achieves this by combining global tokens for overall context, local tokens for neighboring interactions, and random tokens to introduce additional global context [2]. Another technique, document segmentation and hierarchical processing, divides documents into smaller segments, with local information processed first. Hierarchical models then aggregate this information for final classification. This approach helps overcome the length limitations of transformer-based models [14]. Additionally, model compression techniques, such as pruning, quantization, and knowledge distillation, are used to reduce memory usage. Pruning involves removing redundant neurons and layers, while quantization lowers the precision of model weights, reducing memory requirements without significantly compromising accuracy. Knowledge distillation transfers the knowledge from a large model to a smaller, more efficient one, retaining much of the original performance while reducing complexity [7].

Multi-task learning is another promising technique where models are trained on related tasks simultaneously, such as classification and summarization. This approach helps improve generalization and reduces the need for large, specialized models for each task [16]. The improved TF-IDF algorithm addresses limitations in traditional TF-IDF, such as poor term discrimination and instability in multi-category datasets, by refining the inverse document frequency (IDF) calculation. This refined IDF calculation enhances classification accuracy and stability by considering the relationships between feature terms across different document categories [19]. Other techniques, like passage and token models, improve the alignment of full-text books with human-written summaries, facilitating better processing of key sections. These models reduce computational overhead by focusing on essential content and improving long document classification [14].

Byte-level text classification, employed in tasks like spam detection and fraud identification, demonstrates the practical application of efficient transformers for managing long documents in real-world scenarios. Unlike character-level models, byte-level classification requires reasoning through unsegmented data, adding complexity [12]. Pre-layer normalization, which moves layer normalization before the attention mechanism, improves training efficiency by stabilizing the training process and allowing the model to handle long documents more effectively [18]. Hybrid models, which combine CNNs and LSTMs or other RNN architectures, capture both local and global context, improving performance in long document classification by managing complex dependencies [20]. LSTM, with its ability to capture long-term dependencies, is particularly effective for long documents, retaining important context

from earlier sections while processing later ones [21,22]. Unsupervised learning with autoencoders, including Variational Autoencoders (VAEs), allows models to learn sentence representations without labeled data, enhancing classification tasks by capturing semantic and syntactic properties [20]. Adversarial training, aimed at improving a classifier's generalization by making it more robust to adversarial examples, is another technique that enhances performance, particularly in text classification tasks [19]. Finally, discriminative active learning (DAL) selects the most informative instances for labeling, reducing the number of labeled examples needed while maximizing model performance, particularly in high-dimensional text classification tasks [24].

CHAPTER 3
SOFTWARE REQUIREMENTS
SPECIFICATION

3.1 Assumptions and Dependencies

Assumptions:

- It is assumed that the system will be provided with high-quality, labeled datasets for training the model. Without such datasets, the model will not be able to classify long documents accurately.
- The users understand that the system can only classify documents that are similar to the types of data it has been previously trained on. Any new or unseen document types may result in inaccurate classifications.
- The system assumes that all documents provided for classification will be preprocessed properly (e.g., cleaned text without formatting issues), as the classification accuracy depends heavily on the quality of the input data.
- Users are expected to have access to computational resources (e.g., cloud-based GPUs) required to run the trained models, since the system itself does not come with an interface or built-in data processing capabilities.

Dependencies:

- The performance of the system relies on the quality and comprehensiveness of the datasets used for training. The classification model cannot handle or generalize to document types not included in the training data.
- The system depends on the use of pre-trained deep learning models and requires adequate tuning and adaptation to the specific long document classification tasks at hand.
- High-performance computing resources, such as cloud GPUs, are required for both training and deploying the model efficiently, as the project involves deep learning-based techniques which can be computationally intensive.
- The classification outcomes are dependent on the data preparation and preprocessing steps. If the input data is not well-structured, it can adversely affect the model's classification accuracy.
- The ability of the system to handle large-scale documents is contingent on its training with extensive, diverse datasets that represent the variety of long documents it will be required to classify.

3.2 Functional Requirements

The functional requirements define the key operations of the system. The system will provide two core functionalities: soil classification and crop recommendation.

3.2.1 System Feature 1 (Long Document Classification)

- The system shall take labeled training data as input to classify documents into predefined categories.
- The system shall process the labeled training data to output the appropriate category based on the data it has been trained on.
- The system shall provide the classification result only for document types that match the labeled data used during the training phase.

3.2.2 System Feature 2 (Pre-Trained Model Utilization)

- The system shall utilize pre-trained deep learning models for document classification, relying on data previously used during training.
- The system shall only classify documents similar to the types it has been trained on, ensuring reliable categorization within the trained domains.
- The system shall provide a notification to the user if the input document type does not match the categories used in model training.

3.3 External Interface Requirements

3.3.1 User Interfaces

- The system will utilize Streamlit to create a hosted user interface. Users will be able to upload one or multiple long documents for classification.
- The system will classify the uploaded documents into one of the pre-defined labels based on the model's training.
- The user interface will also report the time taken for the classification process.

3.3.2 Hardware Interfaces

- The system will require hardware with sufficient processing power, such as high-performance computing systems or cloud-based servers, to handle the model's training and classification tasks.
- GPUs or TPUs may be needed to optimize the processing of deep learning models for document classification.

3.3.3 Software Interfaces

1. Operating System: Anyone among MAC OSX, Linux or Windows
2. Jupyter Notebook
3. Dataset pertinent to medical images
4. Programming Language: Python

3.3.4 Communication Interfaces

1. RESTful APIs: HTTP requests to retrieve csv datasets.
2. Transformers library: Uses APIs to fetch pretrained models from Hugging Face.

3.4 Nonfunctional Requirements

3.4.1 Performance Requirements

- The system must process labeled training data and classify documents into predefined categories within a reasonable time frame, depending on the dataset size and model complexity.
- The system should be capable of handling the classification of long documents without significant delay.
- The system should handle multiple classification requests for different datasets concurrently without performance degradation.

3.4.2 Safety Requirements

- The system should ensure that the labeled data used for training is accurate and free from any corrupt or malicious content to prevent incorrect classifications.

3.4.3 Security Requirements

- Use HTTPS Connections: `pandas.read_csv` over HTTPS ensures secure communication between your machine and the remote server, preventing man-in-the-middle (MITM) attacks.
- Environment Isolation: Use virtual environments (e.g., `venv`, `Conda`) to isolate dependencies and prevent conflicts or security issues.
- Regular security audits should be conducted to identify and fix vulnerabilities.

3.4.4 Software Quality Attributes

- The system should be highly available and resilient to failures, ensuring minimal downtime.
- The system must be maintainable, with clear documentation and a modular structure to facilitate future updates and debugging.
- The system should be scalable to handle an increasing amount of data or users without performance degradation.

3.5 System Requirements

3.5.1 Database Requirements

- The system should utilize a robust database to store training data, model parameters, and classification results.
- The database should be optimized for fast retrieval of documents and their corresponding classification results.

3.5.2 Software Requirements (Platform Choice)

- The system will be developed using Python and should use Jupyter notebooks, Google Collab, or similar development environments for model training and testing.
- External libraries such as NumPy, Pandas, and Sci-kit Learn will be used for data preprocessing and handling.

3.5.3 Hardware Requirements

- The system requires hardware capable of running deep learning models, preferably equipped with a GPU for faster training and classification tasks.

3.6 Analysis Models: SDLC Model to be applied

The Agile Software Development Life Cycle (SDLC) will be applied to this project to allow for continuous improvement and adaptation throughout the development process. Agile's iterative approach will help refine the system based on feedback from users and stakeholders.

CHAPTER 4

SYSTEM DESIGN

4.1 System Architecture

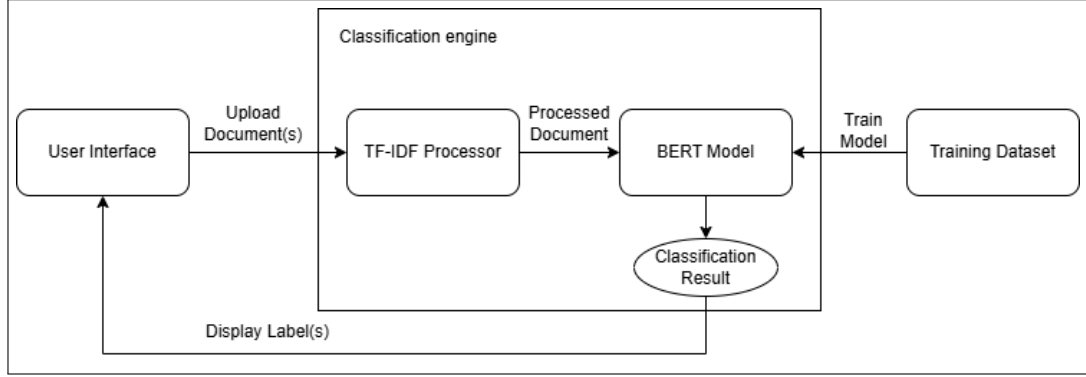


Figure 4.1: System Architecture

4.2 Mathematical Model

The proposed sentence-ranking approach for long document classification is grounded in **TF-IDF (Term Frequency-Inverse Document Frequency)** scoring, which quantifies the importance of sentences based on their term distributions. Unlike random sentence selection, this method ensures that the most informative content is retained, optimizing both computational efficiency and classification accuracy.

4.2.1 Sentence Scoring Mechanism

Each sentence S_i in a document is assigned a **cumulative TF-IDF score**, computed as the sum of the TF-IDF values of its constituent terms t_j :

$$Score(S_i) = \sum_{t_j \in S_i} TF - IDF(t_j) \quad (4.1)$$

where:

- **Term Frequency (TF)** measures how often a term t_j appears in S_i , normalized by sentence length:

$$TF(t_j) = \frac{Frequency\ of\ t_j\ in\ S_i}{Total\ terms\ in\ S_i} \quad (4.2)$$

- **Inverse Document Frequency (IDF)** penalizes terms that appear frequently across sentences, emphasizing rare but discriminative terms:

$$IDF(t_j) = \log \left(\frac{N}{1 + \text{Sentence frequency of } t_j} \right) \quad (4.3)$$

Here, N is the total sentences in the document, and the denominator avoids division by zero.

4.2.2 Sentence Ranking and Selection

Sentences are ranked in descending order of their scores, ensuring high-relevance content is prioritized. The ranked list $\mathcal{R} = [S_1, S_2, \dots, S_N]$ is then truncated using one of three strategies:

1. **Fixed-length selection:** Retain top- k sentences.
2. **Percentage-based selection:** Retain top- $x\%$ of sentences.
3. **Weighted ranking:** Adjust scores by sentence length to balance brevity and informativeness.

4.2.3 Advantages Over Baseline Methods

- **Semantic Relevance:** TF-IDF captures term distinctiveness, preserving context-critical sentences.
- **Computational Efficiency:** By reducing input length, training/inference time decreases without sacrificing accuracy.
- **Scalability:** Adapts to any document length or language (e.g., Indic NLP tokenizer for Marathi).

This mathematical framework ensures systematic input optimization for transformer models like *marathi-bert-v2*, enabling efficient long-document classification.

4.3 Data Flow Diagrams

4.3.1 Level 0 DFD

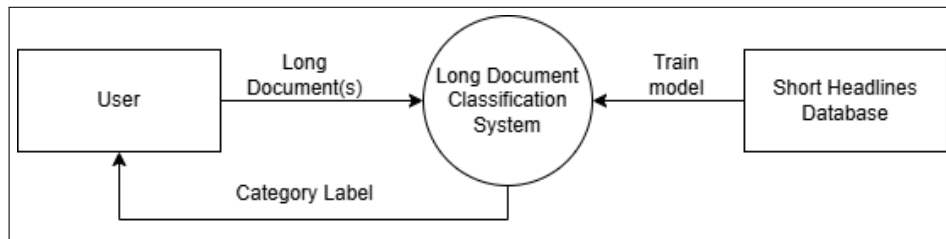


Figure 4.2: Level 0 Data Flow Diagram

4.3.2 Level 1 DFD

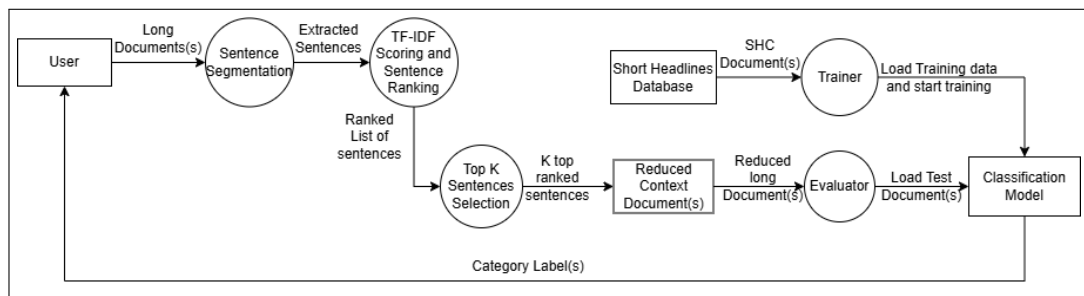


Figure 4.3: Level 1 Data Flow Diagram

4.4 Entity Relationship Diagram

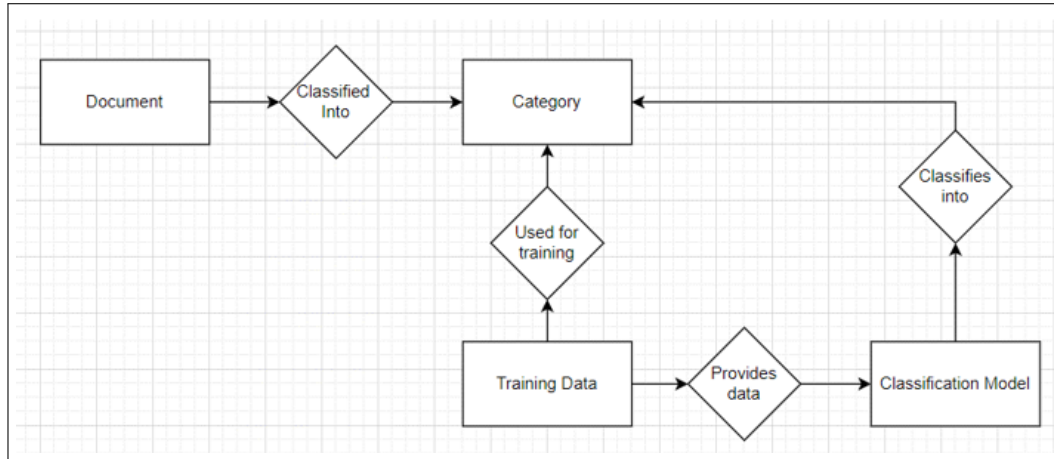


Figure 4.4: Entity Relationship diagram

4.5 UML Diagrams

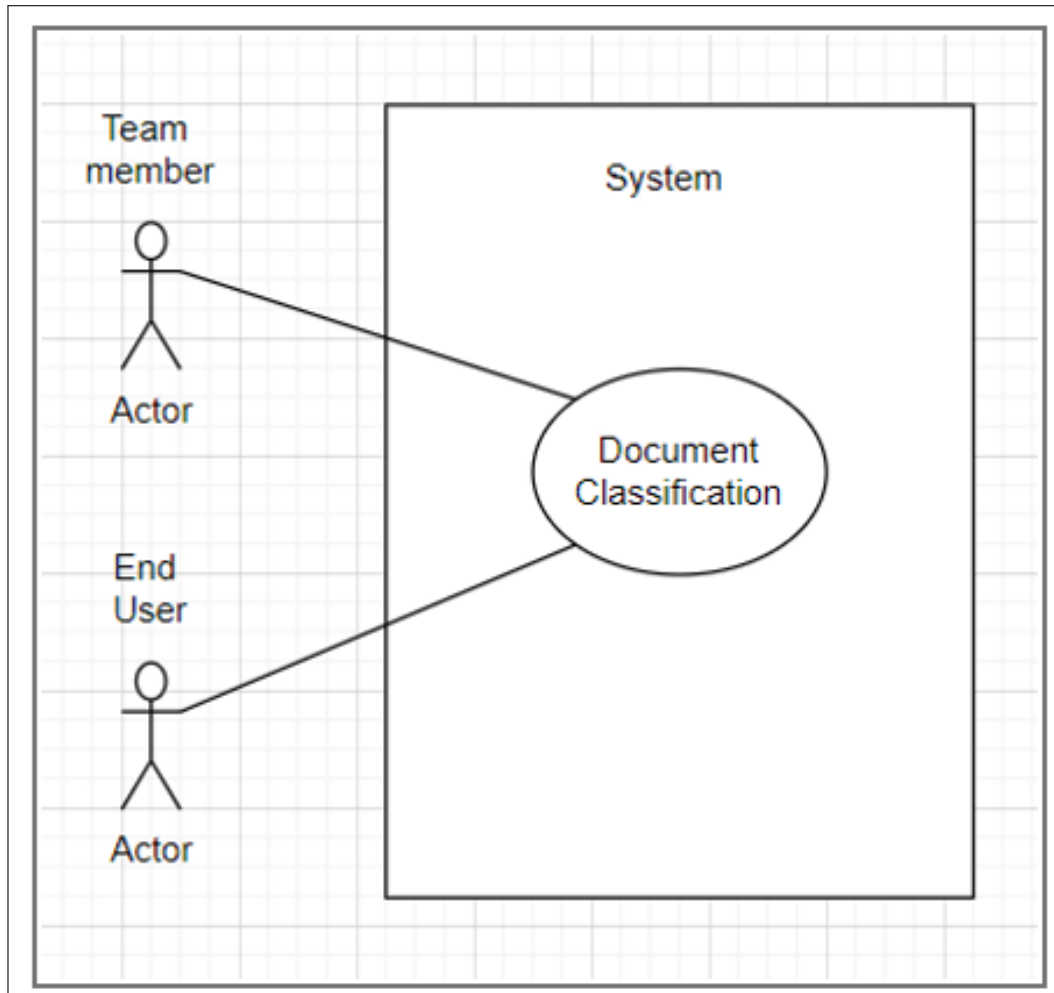


Figure 4.5: Use case diagram

Improving the efficiency of Deep Learning Based Long Document Classification

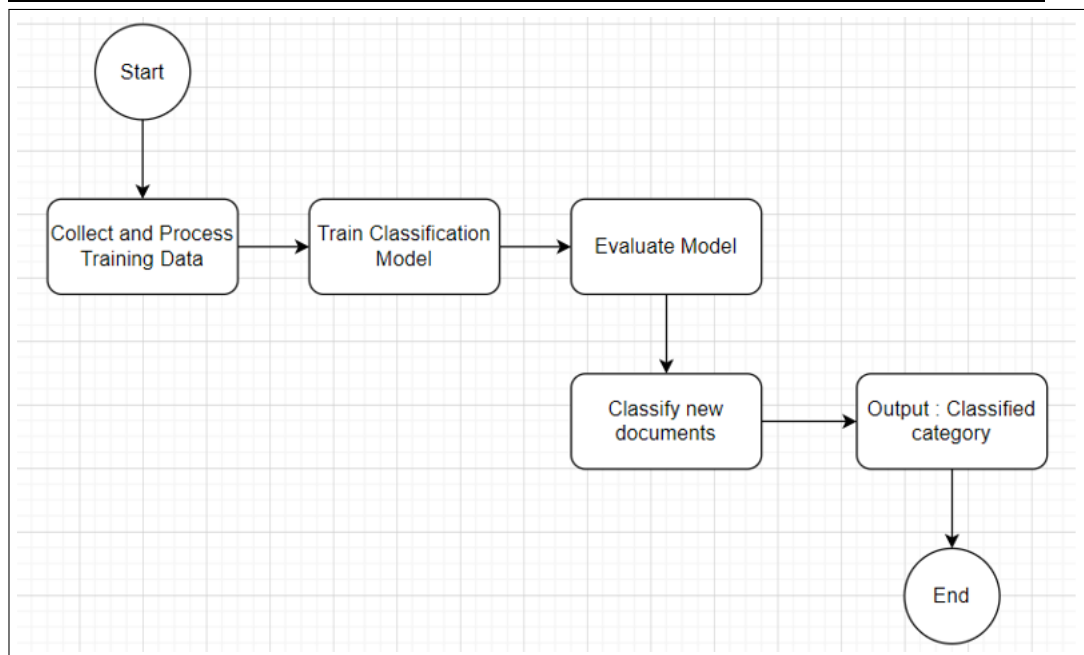


Figure 4.6: Activity Diagram

4.6 Sequence Diagram

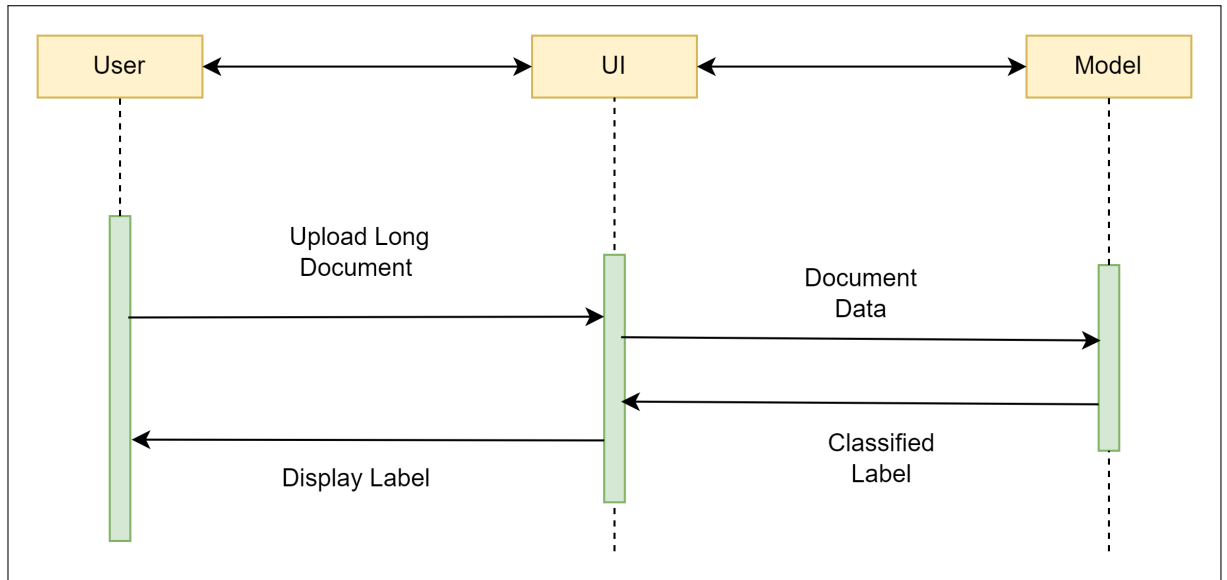


Figure 4.7: Sequence Diagram

CHAPTER 5

PROJECT PLAN

5.1 Project Estimate

5.1.1 Reconciled Estimates

5.4.3 Reconciled Elements

- **Development Hours:** Approximately 500 hours.
- **Cloud Costs:** \$0 — All experiments were conducted using Google Collab (Free tier).
- **Dataset:** MahaNews dataset.
- **Tools Used:** Google Colab, Google Docs, Google Meet, WhatsApp.
- **Total Budget:** \$0 — No monetary costs incurred; all tools and resources used were free or educational.

5.1.2 Project Resources

- **Team:** 3 Computer Science students (roles rotated)
- **Hardware:** Free-tier cloud (Google Collab)
- **Software:** Google Collab (Free)
- **Libraries:** NumPy, Pandas, collections, statistics, NLTK, re, indicnlp, Scikit-learn, Streamlit, ngrok

5.2 Risk Management

Effective risk management is crucial for ensuring the success of the project by identifying, analyzing, and addressing potential risks that could hinder progress. This section outlines the key risks, analysis, and mitigation strategies for the project Improving the Efficiency of Deep Learning-Based Long Document Classification Models.

5.2.1 Risk Identification

- **Computational Limitations:** Processing long documents requires significant computational resources. Limitations in hardware or access to high-performance computing could slow down model training and experimentation.
- **Model Accuracy and Efficiency Trade-offs:** Improving model efficiency without sacrificing accuracy is a challenge. Balancing the two could be difficult, potentially leading to less effective classification models.
- **Data Handling and Availability:** Accessing and managing large-scale datasets necessary for training long document classification models could be a bottleneck, especially if the data is incomplete or poorly structured.
- **Integration and Scalability Issues:** Incorporating optimized models into larger systems or applications may present difficulties, especially when scaling to handle a high volume of long documents.
- **Software or Technical Failures:** Development tools, libraries, or environments could fail unexpectedly, leading to potential delays in development.

5.2.2 Risk Analysis

- **Computational Limitations:** Medium likelihood, high impact. Insufficient resources could drastically slow model training and reduce experimentation capabilities.
- **Model Accuracy and Efficiency Trade-offs:** High likelihood, medium impact. Difficulty in maintaining accuracy while improving efficiency could affect the overall success of the project.

- **Data Handling and Availability:** Medium likelihood, medium impact. Poor-quality data or insufficient volumes could result in ineffective model training, impacting both efficiency and accuracy.
- **Integration and Scalability Issues:** Medium likelihood, medium impact. Delays in integrating and scaling models could slow deployment and affect system performance.
- **Software or Technical Failures:** Low likelihood, medium impact. These failures could cause temporary delays but are generally manageable with proper planning and backups.

5.2.3 Overview of Risk Mitigation, Monitoring, Management

- **Computational Limitations:** Explore cloud-based platforms (e.g., AWS, Google Cloud) for additional computational power. Optimize models to reduce resource consumption and enable efficient training on available hardware.
- **Model Accuracy and Efficiency Trade-offs:** Use model tuning techniques, such as hyperparameter optimization and regular validation, to maintain a balance between accuracy and efficiency. Experiment with architectures like Longformer to enhance efficiency without compromising performance.
- **Data Handling and Availability:** Leverage publicly available long document datasets and use data preprocessing techniques to clean and structure data. If needed, data augmentation methods can be applied to increase the dataset size and improve training outcomes.
- **Integration and Scalability Issues:** Use modular approaches during development to integrate optimized models in smaller, testable units. Perform stress testing to ensure scalability across various environments and volumes of documents.
- **Software or Technical Failures:** Regularly back up all code and data. Maintain redundancy in development environments and use version control systems to track changes and restore functionality if issues arise. Continuous monitoring will be employed to quickly identify and address technical issues.

5.3 Project Schedule

5.3.1 Project Task Set

- Literature review on long document classification and sentence selection techniques.
- Preprocessing and preparation of the MahaNews dataset.
- Implementation of TF-IDF-based sentence ranking and extraction.
- Model training and evaluation.
- Development of a user interface to upload documents and show predicted category.
- Documentation and report writing of all findings and experiments.

5.3.2 Task Network

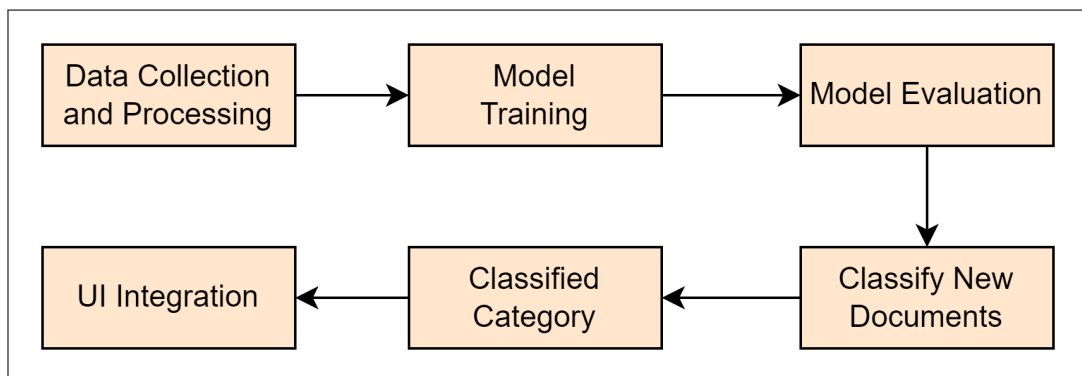


Figure 5.1: Task Network

5.3.3 Timeline Chart

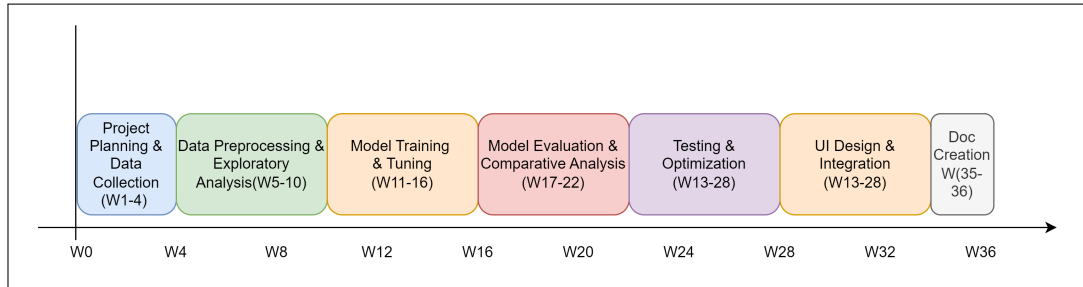


Figure 5.2: Timeline Chart

5.4 Team Organization

5.4.1 Team structure

- **Member 1 – Data and Model Lead:** Handles dataset preprocessing, implements sentence selection strategies (truncation, random, TF-IDF), and trains the marathi-bert-v2 model.
- **Member 2 – Evaluation and Results Analyst:** Evaluates model performance across strategies, conducts SHC-to-LDC adaptation analysis, and documents experimental results and observations.
- **Member 3 – UI and Project Coordinator:** Develops the user interface for input and output visualization, manages overall progress, ensures integration of all components, and prepares general documentation.

5.4.2 Management reporting and communication

- **Weekly Meetings:** Internal team meetings were conducted every week about an 1 hour to review progress, discuss challenges, and plan next steps.
- **External Guide Meetings:** Weekly 30-minute sessions were held with the external guide to receive feedback and technical guidance.

- **Progress Reporting:** Bi-weekly progress updates were shared with the department to ensure transparency and alignment with academic requirements.
- **Communication Tools:** A WhatsApp group was maintained for daily coordination and quick queries, while Google Meet was used for all virtual discussions.
- **Development and Documentation Tools:** Google Collab was used for implementing and testing models, while Google Docs was used for collaborative documentation and report writing.

CHAPTER 6

PROJECT IMPLEMENTATION

6.1 Overview of Project Modules

6.1.1 Dataset Collection and Preprocessing

To prepare and preprocess data for experimentation, the MahaNews dataset will be collected and cleaned to ensure quality and relevance. The preprocessing phase involves tokenizing documents into smaller units and computing sentence-level features, such as TF-IDF scores, to capture their significance. Additionally, documents will be segmented into sentences to enable more granular ranking and selection strategies, ensuring the data is structured for effective experimentation and analysis.

6.1.2 Sentence Selection Strategies

The objective is to develop and evaluate various sentence selection techniques aimed at reducing the input size while preserving critical information. These techniques include truncation, which involves selecting sentences from the beginning or end of the document, and random selection, where sentences are randomly chosen for input representation. Additionally, ranking-based selection leverages TF-IDF scores to rank and select the most informative sentences. The evaluation of these strategies focuses on understanding their impact on classification performance and computational efficiency, providing insights into the trade-offs between input reduction and model effectiveness.

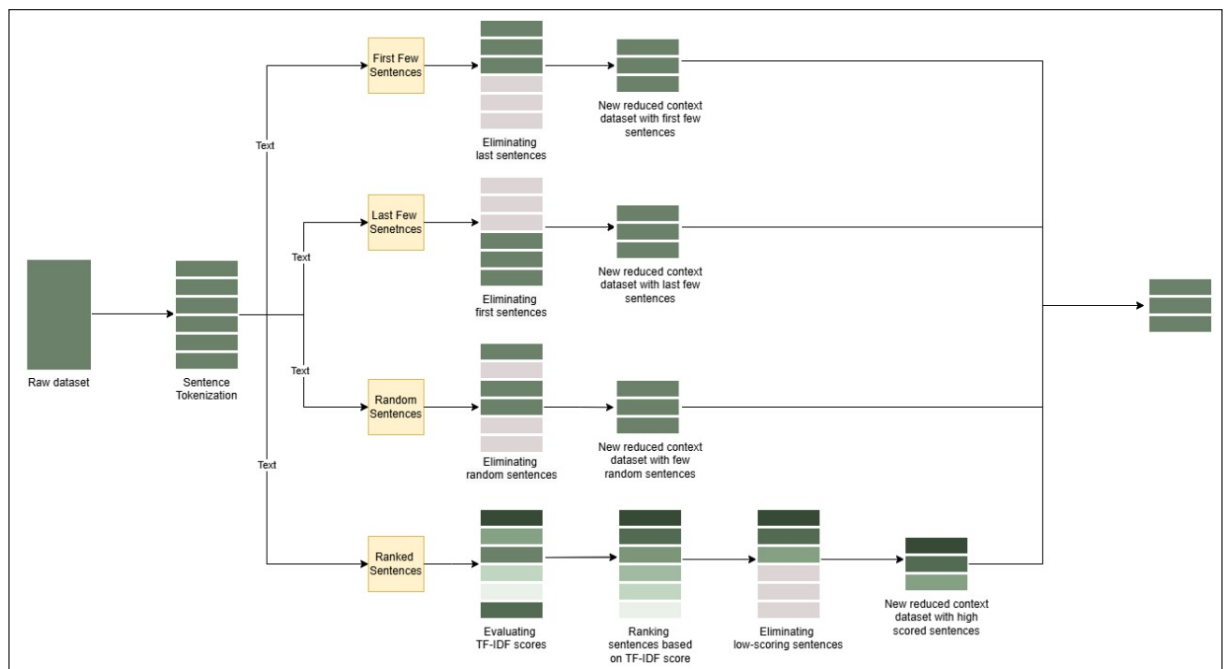


Figure 6.1: Sentence Selection Diagram

6.1.3 Model Development and Adaptation

The objective is to build a classification model by leveraging the selected sentence subsets to enhance performance. This involves fine-tuning the marathi-bert-v2 model specifically for Long Document Classification (LDC) tasks to align with the linguistic and structural nuances of the dataset. Ranked sentence selection will be seamlessly integrated into the model pipeline to ensure that only the most informative content contributes to the classification process. Additionally, advanced techniques from Short Headlines Classification (SHC) document classification are adapted to the LDC context, optimizing the model’s ability to handle lengthy documents effectively while maintaining computational efficiency.

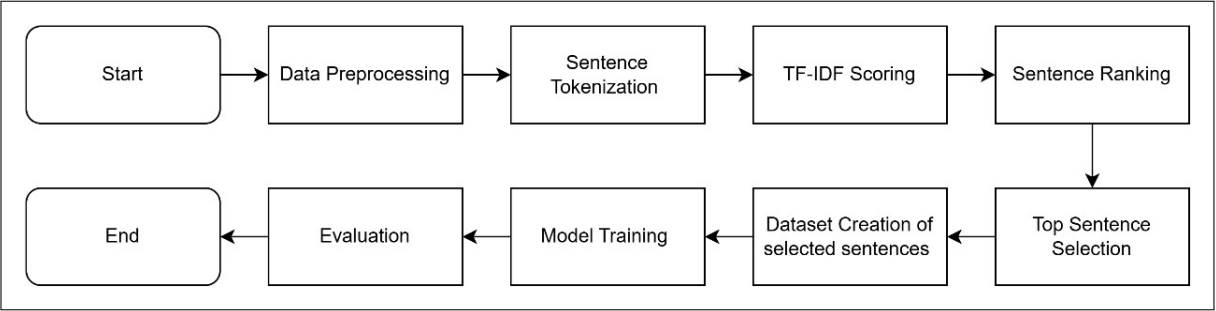


Figure 6.2: Process Diagram

6.1.4 Cross-Domain Adaptation

The objective is to validate the effectiveness of adapting Short Headline Classification (SHC) document classification techniques to the Long Document Classification (LDC) context. This involves testing the proposed approach on both short and long document datasets to evaluate its versatility and applicability across different types of textual data. A key focus is on analyzing the model's generalization capability across domains, ensuring its robustness and adaptability. Furthermore, the validation process examines how ranked sentence selection contributes to mitigating computational overhead without compromising the accuracy or reliability of the classification, demonstrating the efficiency of the adaptation techniques.

6.2 Tools and Technologies Used

Category	Technologies
Frontend	Streamlit
Backend	Python
Data Processing	Pandas, NumPy
NLP, Machine Learning	NLTK, SimpleTransformers
APIs	SimpleTransformers

Table 6.1: Technologies Used in the Project

6.3 Algorithm Details

6.3.1 Document Classification

1. Preprocess the SHC and LDC Datasets

- Load SHC dataset for training and LDC dataset for testing.
- Preprocess the text by:
 - Cleaning unnecessary characters and formatting.
 - Tokenizing the text into words.
 - Splitting LDC documents into individual sentences.

2. Compute TF-IDF Scores for LDC Dataset

- Compute term frequencies (TF) for each sentence in the LDC dataset.
- Compute inverse document frequency (IDF) across all documents in the LDC dataset.
- Calculate TF-IDF scores by aggregating word-level scores for each sentence.
- Rank sentences by their TF-IDF scores and select the highest-ranked sentence.

3. Train BERT Model on SHC Dataset

- Prepare training data by:
 - Tokenizing sentences from the SHC dataset.
 - Generating attention masks for the tokenized sentences.
- Fine-tune the BERT model on the SHC dataset using a classification loss function.

4. Test on LDC Dataset Using Highest-Ranked Sentence

- For each document in the LDC dataset:
 - Select the highest-ranked sentence based on TF-IDF scores.
- Classify the selected sentence using the fine-tuned BERT model.

6.3.2 Sentence Selection

1. Calculate TF-IDF Scores for Each Sentence

- Compute the TF-IDF score for each word in the sentence.
- Take the average of the word-level TF-IDF scores and assign it as the sentence's overall TF-IDF score.

2. Rank the Sentences

- Rank all sentences in the document based on their overall TF-IDF scores in descending order.

3. Determine the Percentage of Context to Select

- Based on the available dataset, fix the percentage of context to be selected from the document (e.g., top 10% of the sentences).

4. Select the Top Ranked Sentences

- From the ranked list of sentences, select the top percentage as determined in the previous step.

CHAPTER 7

SOFTWARE TESTING

7.1 Type of Testing

1. Functional Testing: Verify that the proposed sentence selection algorithm performs as expected by accurately ranking and selecting sentences.
2. Performance Testing: Measure the computational efficiency of the model, including memory usage and inference time, when processing long documents.
3. Cross-Domain Adaptation Testing: Evaluate the model's ability to adapt from SHC training data to long document classification (LDC) tasks.
4. Regression Testing: Ensure that modifications to the sentence selection algorithm or the model do not negatively impact overall classification performance.

7.2 Test cases & Test Results

- **TC1:** Verify the calculation of TF-IDF scores for sentences.
 - * **Description:** Ensure correct computation of TF-IDF scores for each sentence.
 - * **Test Result:** Correct TF-IDF scores are computed for each sentence.
- **TC2:** Validate the ranking of sentences based on TF-IDF scores.
 - * **Description:** Check that sentences are ranked correctly in descending order of TF-IDF scores.
 - * **Test Result:** Sentences are ranked correctly in descending order of TF-IDF scores.
- **TC3:** Test classification accuracy using selected top-ranked sentences.
 - * **Description:** Evaluate classification accuracy with the selected top-ranked sentences.
 - * **Test Result:** Classification accuracy with reduced context (50%): 79.53%. Full context accuracy: 80.314%.
- **TC4:** Measure computational efficiency during testing.
 - * **Description:** Assess computational efficiency, such as testing time.
 - * **Test Result:** Testing time reduced significantly to 21.9s.
- **TC5:** Evaluate cross-domain adaptation (SHC to LDC).
 - * **Description:** Test the model's ability to adapt from SHC to LDC tasks.
 - * **Test Result:** Comparable accuracy between reduced context (79.53%) and full context (80.314%).

CHAPTER 8

RESULTS

8.1 Outcomes

The system was evaluated across two key performance dimensions:

8.1.1 Model Accuracy

To evaluate the effectiveness of our context reduction strategies for long document classification, we conducted experiments across multiple configurations. The performance was primarily measured in terms of classification accuracy under different sentence selection methods and pre-processing approaches. The following tables present the classification accuracy obtained using different sentence selection strategies which are then compared with the best-performing naive reduction strategies.

8.1.1.1 Fixed Number of Sentences Selection

Number of Sentences	Accuracy
1 Sentence	75.45%
2 Sentences	77.64%
3 Sentences	76.90%
4 Sentences	76.82%
5 Sentences	79.84%

Table 8.1: Accuracy using top-k ranked sentences (Raw TF-IDF)

This table shows the classification accuracy when selecting the top-k sentences from the document using raw TF-IDF scores. Accuracy improves as more sentences are selected, peaking at 5 sentences with 79.84%, indicating effective information retention through ranking.

8.1.1.2 Fixed Number of Sentence Selection using Normalized TF-IDF

Number of Sentences	Accuracy
1 Sentence	73.45%
2 Sentences	77.96%
3 Sentences	78.40%
4 Sentences	79.84%
5 Sentences	79.84%

Table 8.2: Accuracy using top-k ranked sentences (Normalized TF-IDF)

Improving the efficiency of Deep Learning Based Long Document Classification

This approach normalizes TF-IDF values per document. It outperforms raw TF-IDF when 2 or more sentences are selected. Accuracy reaches 79.84% with 4 or 5 sentences, matching the best performance while using fewer tokens.

8.1.1.3 Percent-Based Selection

Percent Selected	Accuracy
75%	80.2%
50%	79.53%
25%	78.31%

Table 8.3: Accuracy with document content selected by percentage

This method selects a fixed percentage of each document instead of ranking sentences. Retaining 75% yields the highest accuracy (80.2%), slightly better than the top-ranked sentence methods, though at the cost of processing more data.

8.1.1.4 Best Naive Approach Results

Naive Approach	Best Accuracy
Drop Last Part (25%)	79.92%
Drop First Part (25%)	78.75%
Drop Random Part (25%)	79.68%
Select Last 5 Sentences	74.96%
Select Random 5 Sentences	74.13%

Table 8.4: Best results from naive context reduction methods

Comparison: Among naive methods, dropping the last 25% of content gives the highest accuracy (79.92%), slightly higher than rank-order methods. However, such approaches are less interpretable and risk discarding critical information.

In comparison, rank-order TF-IDF strategies achieve nearly the same performance (up to 79.84%) using only 4-5 selected sentences. This makes them more efficient and semantically informed for long document classification, offering an optimal balance between accuracy and reduced input size.

8.1.2 Inference Time

Number of Sentences	Inference Time
1 Sentence	21.9s
2 Sentences	23.0s
3 Sentences	21.9s
4 Sentences	24.0s
5 Sentences	23.7s

Table 8.5: Inference time per sample for varying number of input sentences

This table presents the inference time required to classify a document when varying the number of input sentences, using a custom `max_seq_length` setup.

Inference time remains relatively stable between 21.9s and 24s per document. Notably, 1 and 3 sentences result in the fastest inference at 21.9s, while 4 sentences lead to the highest delay (24s). This suggests that selecting fewer informative sentences can both improve efficiency and maintain accuracy, especially when using TF-IDF-based sentence ranking.

While naive approaches may retain more content, they are generally less efficient and require longer sequences, potentially increasing inference time and memory usage. The TF-IDF-based approach provides a reliable trade-off between accuracy and computational performance, making it suitable for large-scale document classification tasks.

8.2 System Screen Shots

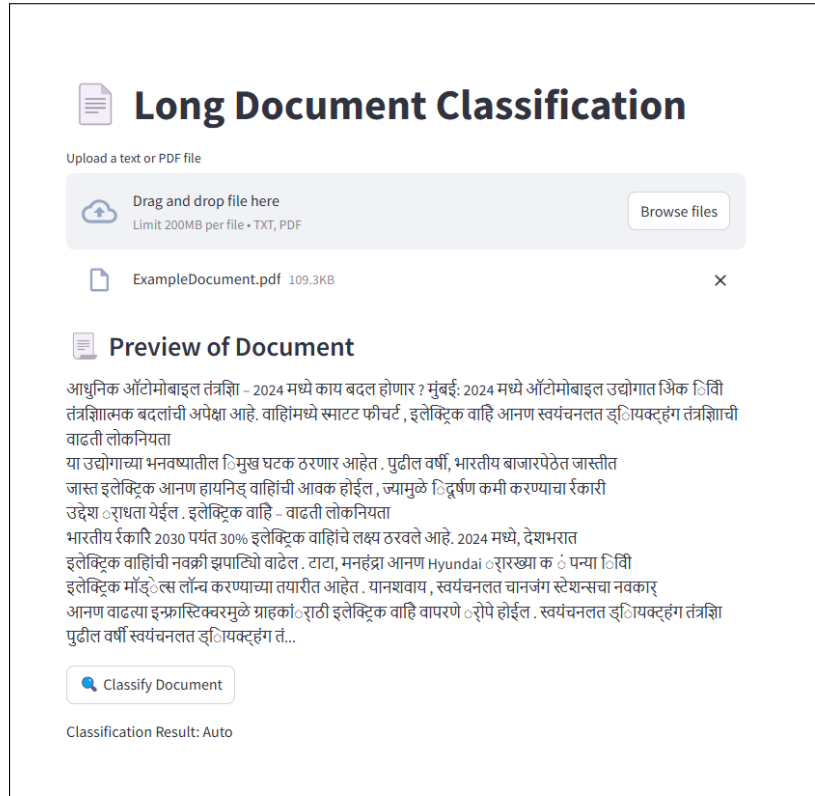


Figure 8.1: User Interface

CHAPTER 9

CONCLUSIONS

9.1 Technical Contributions

1. TF-IDF-Based Sentence Ranking:

- Developed a novel approach to rank sentences in long documents using TF-IDF scores.
- Demonstrated that ranking and selecting high-scoring sentences effectively condenses document context while retaining essential information.

2. Context Reduction Without Accuracy Loss:

- Achieved a significant reduction in document length (50% context) while maintaining classification accuracy comparable to full context (79.53% vs. 80.314%).
- Highlighted the trade-off between computational efficiency and model performance.

3. Cross-Domain Adaptation:

- Successfully adapted a model trained on Short Headlines Content (SHC) documents to classify Long Document Content (LDC).
- Validated the transferability of knowledge from one domain to another with minimal performance degradation.

4. Enhanced Computational Efficiency:

- Reduced testing time to 21.9 seconds by limiting the input size without compromising model accuracy.
- Demonstrated the feasibility of the approach for real-time applications.

9.2 Limitations and Challenges

1. Limited Generalizability:

- The approach was tested on a single dataset (MahaNews), and its generalizability to other datasets or languages remains uncertain. Different datasets may have different characteristics, which could affect the performance of the proposed method.

2. Dependency on TF-IDF:

- The reliance on TF-IDF for sentence ranking might not always yield the most semantically meaningful sentences. TF-IDF is a statistical measure that may miss context-dependent nuances or domain-specific terms that are important for accurate classification.

3. Model Adaptation Complexity:

- The model's ability to adapt to different document lengths and domains requires careful tuning of hyperparameters. This adaptation can be computationally expensive and may require substantial domain-specific adjustments for optimal performance.

4. Contextual Sentence Selection Limitations:

- The method assumes that important information is concentrated in certain sentences, but this may not hold true for every document. Documents with evenly distributed information may not benefit from the ranking and selection approach as much as those with clear, information-dense sections.

9.3 Future Work

1. Cross-Domain Generalization and Transfer Learning:

- Expanding the approach to handle a wider variety of domains (e.g., legal, medical, and technical documents) is critical for its practical application. This can be done by employing transfer learning, where models trained on one domain (e.g., SHC documents) are fine-tuned to work in other, more complex domains (e.g., long documents).

2. Optimization for Large-Scale Real-Time Applications:

- A significant challenge moving forward is to optimize the model for scalability and real-time performance. Techniques such as model pruning, quantization, or the development of lightweight transformer models (e.g., DistilBERT) could be explored to reduce both computational time and resource consumption while maintaining accuracy.

3. Evaluation on Multilingual and Multimodal Datasets:

- Extending the approach to multilingual datasets will increase the model's robustness and applicability across different languages. Additionally, incorporating multimodal data, such as images, graphs, or tables, into the document classification process can provide richer context and improve classification accuracy in documents that mix text with other media.

9.4 Applications

1. Document Classification:

- The proposed method can be applied to classify long documents in various domains, including news articles, research papers, legal documents, and medical records. By selecting the most informative sentences, the method helps reduce the computational overhead while preserving the document's classification accuracy.

2. Cross-Domain Knowledge Transfer:

3. Search Engines:

- By improving the ranking of informative sentences, this method can be used to enhance search engine results by providing concise summaries of relevant documents. This can lead to faster and more accurate search results, especially for long documents.

4. Multilingual and Cross-Cultural Applications:

- The method can be extended to handle multilingual documents, enabling applications such as cross-lingual document classification, translation, and cross-cultural sentiment analysis. This would help bridge the language and cultural gaps in global information systems.

5. Personalized Content Recommendation:

- By efficiently identifying the key sentences in user-generated content (such as blog posts, articles, or reviews), this method can be utilized in content recommendation systems to provide users with personalized and relevant content based on their preferences and browsing history.

6. Academic Research and Citation Analysis:

- The approach can be applied to academic research where long papers need to be classified or summarized. By reducing the document context without losing critical information, this method can help in citation analysis, academic search engines, and identifying influential research papers in a specific field.

7. Legal and Regulatory Document Analysis:

- This approach can be beneficial in the legal domain, where long legal documents or contracts need to be analyzed and categorized. By selecting the most relevant sentences, the method can streamline legal document review processes and assist in finding pertinent information quickly.

8. Healthcare Document Processing:

- In healthcare, large medical records and research papers can be efficiently processed using this approach. It can be applied to extract critical information from patient records or research articles, enabling healthcare professionals to make informed decisions without being overwhelmed by the size of the documents.

CHAPTER 10

APPENDIX

10.1 Appendix A

Problem Statement Feasibility Assessment

1. Problem Statement Analysis

The given problem focuses on improving the efficiency of deep learning-based long document classification models. It involves techniques such as sentence ranking, TF-IDF computation, and cross-domain adaptation. To assess feasibility:

- **Satisfiability Analysis:** The problem is considered satisfiable as the goal is well-defined, with measurable objectives such as accuracy, computational efficiency, and adaptability.
- **Key Variables:**
 - Input data characteristics (document length, context density).
 - Sentence ranking and selection mechanisms.
 - Cross-domain transfer learning effectiveness.
- The constraints are manageable given current computational resources and tools (e.g., Python, BERT, Pandas, Streamlit).

2. Computational Complexity

We analyze the problem's complexity classification under NP-Hard, NP-Complete, or P-Type based on its components:

- **TF-IDF Score Computation:**
 - Computing TF-IDF for each sentence involves matrix operations that are polynomial in complexity ($O(n \cdot m)$, where n is the number of documents and m is the number of unique terms).
 - **Classification:** This task belongs to the P class as it is solvable in polynomial time.

- **Sentence Ranking:**
 - Ranking sentences based on TF-IDF scores involves sorting, which has a complexity of $O(k \log k)$ for k sentences.
 - **Classification:** This task is also P-Type.
- **Model Training (Fine-tuning BERT):**
 - Fine-tuning deep learning models is computationally intensive, often requiring iterative optimization. The process is heuristic-driven, but the overall classification can be approximated as NP-Hard due to high computational requirements.
- **Cross-Domain Adaptation:**
 - Adapting a model trained on SHC documents to LDC tasks involves additional challenges like feature distribution differences and domain alignment. This task is computationally challenging but feasible with domain adaptation techniques.
 - **Classification:** Likely NP-Complete due to its reliance on efficient domain transfer with unknown alignment complexity.

3. Modern Algebra and Mathematical Models

Modern algebra provides tools for understanding the transformations and mappings in sentence ranking and domain adaptation:

- **Group Theory:**
 - Sentence ranking based on TF-IDF can be represented as a group operation where ranking functions preserve order invariants under transformations.
- **Vector Space Models:**
 - Document and sentence representations in TF-IDF are points in a vector space. The operations (dot product for cosine similarity, ranking by magnitude) align well with linear algebra principles.
- **Domain Adaptation as a Homomorphism:**
 - Cross-domain adaptation can be viewed as a homomorphism between feature spaces of SHC and LDC, ensuring structural consistency during transfer.

10.2 Appendix B

- **Paper Publication Details:**
 - **Paper Title:** Survey on Improving the Efficiency of Long Document Classification
 - **Conference/Journal Name:** Computer Research and Development Journal (journalcrd)

10.3 Appendix C

Plagiarism Report of project report.

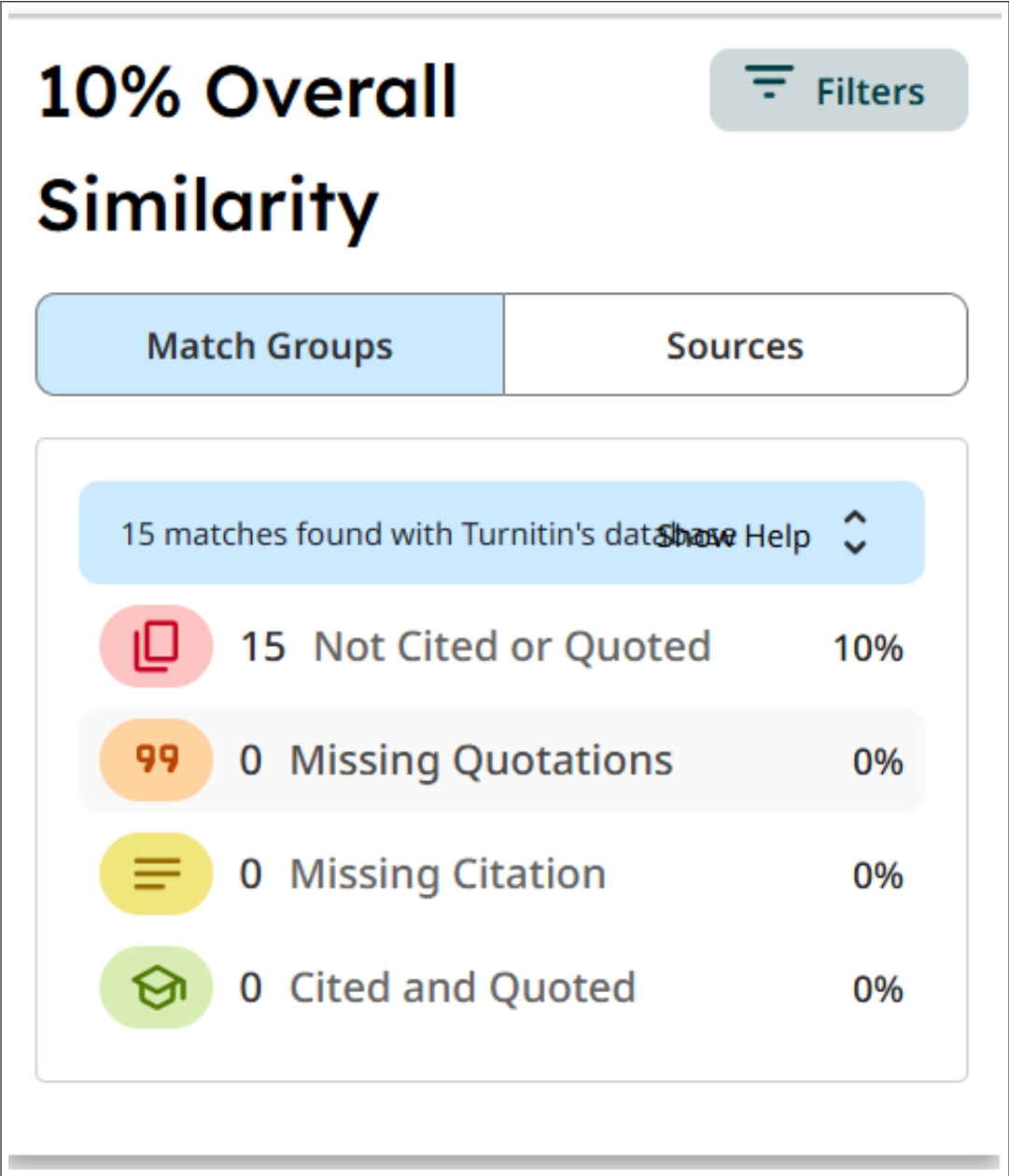


Figure 10.1: Plagiarism Report

CHAPTER 11

REFERENCES

- [1] Park, H.: Efficient classification of long documents using transformers. 2022. 10.18653/v1/2022.acl-short.79
- [2] Zaheer, M., Ainslie, J., Guruganesh, G., et al.: Big Bird: Transformers for longer sequences. Dublin, Ireland; 2020. 2:702-709. 10.48550/arXiv.2007.14062
- [3] Liu, C.-Z., Sheng, Y.-X., Wei, Z.-Q., Yang, Y.-Q.: Research of Text Classification Based on Improved TFIDF Algorithm. Association for Computational Linguistics, 2018. 10 18653 1 2022 - 79. 10.1109/IRCE.2018.8492945
- [4] Song, B.: State Space Models Based Efficient Long Documents Classification. 2024, 16:10.4236/jilsa.2024.163009
- [5] Moro, G.: Efficient Memory-Enhanced Transformer for Long-Document Summarization in Low-Resource Regimes. Sensors, 23. 2023, 3542-3542. 10.3390/s23073542.
- [6] Khandve, S.: Hierarchical neural network approaches for long document classification. 2022. 10.48550/arXiv.2201.06774
- [7] Zafrir, O.: Q8BERT: Quantized 8Bit BERT. Vancouver, BC, Canada; 2019. 36-39, 10.1109/EMC2-NIPS53020.2019.00016
- [8] He, J.: Long Document Classification From Local Word Glimpses via Recurrent Attention Learning. 2019, 7:40707-40718. 10.1109/ACCESS.2019.2907992
- [9] Wagh, V.: Comparative Study of Long Document Classification. 2021; 10.1109/TENCON54134.2021.9707465. 10.1109/TENCON54134.2021.9707465
- [10] Devlin, J., Chang, M. W., Lee, K., et al.: BERT: Pre-training of deep bidirectional transformers for language understanding. 10.48550/arXiv.1810.04805
- [11] Liu, L., Liu, K., Cong, Z., Zhao, J., Ji, Y., He, J.: Long Length Document Classification by Local Convolutional Feature Aggregation. Algorithms. 10.3390/a11080109
- [12] Al-Qurishi, M.: Recent Advances in Long Documents Classification Using Deep-Learning. 2022. 10.18653/v1/2022.icnlp-1.12

- [13] Bamman, D., Smith, N.: New Alignment Methods for Discriminative Book Summarization. Dublin, Ireland; 2:702-709. 10.48550/arXiv.1305.1319
- [14] Tay, Y., Dehghani, M., Abnar, S., et al.: Long Range Arena: A Benchmark for Efficient Transformers. OpenReview. Association for Computational Linguistics, 2021. 10 18653 1 2022 - 79. 10.48550/arXiv.2011.04006
- [15] Mittal, S.: L3Cube-MahaNews: News-Based Short Text and Long Document Classification Datasets in Marathi. 2024, 52-63. 10.1007/978-3-031-58495-4_4
- [16] Mirashi, A.: L3Cube-IndicNews: News-based Short Text and Long Document Classification Datasets in Indic Languages. 2024. 10.48550/arXiv.2401.02254
- [17] Jain, K., Deshpande, A., Shridhar, K., et al.: Indic-Transformers: An Analysis of Transformer Language Models for Indian Languages. 2020, 10.48550/arXiv.2011.02323
- [18] Pham, L. M., Cao The, H.: LNLf-BERT: Transformer for Long Document Classification with Multiple Attention Levels. 2024, 11:10.1109/ACCESS.2024.3492102
- [19] Qaiser, S., Ali, R.: Text Mining: Use of TF-IDF to Examine the Relevance of Words to Documents. 2018, 10.5120/ijca2018917395
- [20] Minaee, S., Kalchbrenner, N., Cambria, E., et al.: Deep Learning Based Text Classification: A Comprehensive Review. 2021, 10.48550/arXiv.2004.03705
- [21] Putri, B. A., Setiawan, E. B.: Topic Classification Using the Long Short-Term Memory (LSTM) Method with FastText Feature Expansion on Twitter. 2023, 10.1109/ICoDSA58501.2023.10277033
- [22] Teragawa, S., Wang, L., Ma, R.: A Deep Neural Network Approach using Convolutional Network and Long Short Term Memory for Text Sentiment Classification. 2021, 10.1109/CSCWD49262.2021.9437871
- [23] Li, C., Cheng, Y., Wang, H.: A Novel Document Classification Algorithm Based on Statistical Features and Attention Mechanism. 2018, 10.1109/IJCNN.2018.8489314

- [24] Prabhu, S., Mohamed, M., Misra, H.: Multi-class Text Classification using BERT-based Active Learning. 2021, 10.48550/arXiv.2104.14289
- [25] Martins, A. F. T., Farinhas, A., Treviso, M., et al.: Sparse and Continuous Attention Mechanisms. 2020, 10.48550/arXiv.2006.07214
- [26] Beltagy, I., Peters, M. E., Cohan, A.: Longformer: The Long-Document Transformer. 2020, 10.48550/arXiv.2004.05150
- [27] Sun, C., Qiu, X., Xu, Y., et al.: How to Fine-Tune BERT for Text Classification? 2020, 10.48550/arXiv.1905.05583
- [28] Das, B., Chakraborty, S.: An Improved Text Sentiment Classification Model Using TF-IDF and Next Word Negation. 2020, 10.48550/arXiv.1806.06407
- [29] Kim, S.-W., Gil, J.-M.: Research paper classification systems based on TF-IDF and LDA schemes. 2019, 10.1186/s13673-019-0192-7
- [30] Das, M., K., Selvakumar, P.J.A. Alphonse: A Comparative Study on TF-IDF feature Weighting Method and its Analysis using Unstructured Dataset. 2023, 10.48550/arXiv.2308.04037
- [31] Yang, Z., Yang, D., Dyer, C., et al.: Hierarchical Attention Networks for Document Classification. 2016, 10.18653/v1/N16-1174

LIST OF ABBREVIATIONS

Abbreviation	Illustration
TF-IDF	Term Frequency- Inverse Document Frequency
BERT	Bidirectional Encoder Representations from Transformers
SHC	Short Headline Classification
LDC	Long Document Classification

LIST OF FIGURES

Figure	Illustration	Page No.
4.1	System Architecture	17
4.2	Level 0 Data Flow Diagram	19
4.3	Level 1 Data Flow Diagram	19
4.4	Entity Relationship Diagram	20
4.5	Use Case Diagram	21
4.6	Activity Diagram	22
4.7	Sequence Diagram	23
5.1	Task Network Diagram	28
5.2	Timeline Chart	29
6.1	Sentence Selection Diagram	33
6.2	Process Diagram	34
8.1	User Interface	46
10.1	Plagiarism Report	57

LIST OF TABLES

Table	Illustration	Page No.
6.1	Tools and Technology Table	36
8.1	Accuracy using top-k ranked sentences Table	43
8.2	Accuracy using top-k ranked sentences (Normalized TF-IDF) Table	43
8.3	Accuracy with document content selected by percentage Table	44
8.4	Best results from naive context reduction methods Table	44
8.5	Inference time per sample for varying number of input sentences Table	45